Тестирование API

Введение

В ходе предыдущих занятий был рассмотрен процесс разработки REST API на платформе ASP.NET Core, а также настройка Swagger (OpenAPI) для автоматической генерации интерактивной документации. Swagger UI предоставляет удобный веб-интерфейс для ознакомления со структурой API и выполнения базовых запросов непосредственно из браузера. Тем не менее в контексте профессиональной разработки и тестирования данный инструмент обладает рядом существенных ограничений.

Ограничения Swagger UI:

- **Отсутствие сохранения запросов.** Каждый раз при обращении к Swagger UI необходимо заново вводить параметры запроса. Инструмент не предоставляет возможности сохранять, именовать и переиспользовать запросы между сессиями.
- **Отсутствие управления окружениями.** Реальный проект, как правило, разворачивается в нескольких средах: локальная разработка (`localhost`) и продуктивная среда (`production`). Swagger UI не поддерживает концепцию окружений — переключение между базовыми URL выполняется вручную.
- **Отсутствие автоматизированных тестов.** Swagger UI предназначен для демонстрации и ручной проверки API. В нём невозможно описать ожидаемое поведение эндпоинта, проверить структуру ответа или зафиксировать результат теста.
- **Отсутствие цепочек запросов.** Типичный сценарий тестирования предполагает последовательность связанных запросов: например, выполнить аутентификацию, получить токен, использовать его в последующих запросах. Swagger UI не поддерживает подобные сценарии.
- **Отсутствие инструментов командной работы.** Swagger — это документация, встроенная в проект. Поделиться набором запросов с коллегой или синхронизировать изменения в рамках команды с его помощью невозможно.

Таким образом, Swagger UI следует рассматривать как инструмент документирования и первичного ознакомления с API, но не как инструмент тестирования. Для решения задач тестирования в профессиональной среде применяются специализированные инструменты, наиболее распространённым из которых является **Postman**.

1. Postman

Postman — это платформа для разработки и тестирования API.

На сегодняшний день Postman является одним из наиболее широко используемых инструментов в области разработки API. Инструмент поддерживает работу с протоколами HTTP/HTTPS, GraphQL, WebSocket, gRPC и др.

Postman распространяется по условно-бесплатной модели: базовый функционал доступен бесплатно, расширенные возможности командной работы предоставляются по подписке.

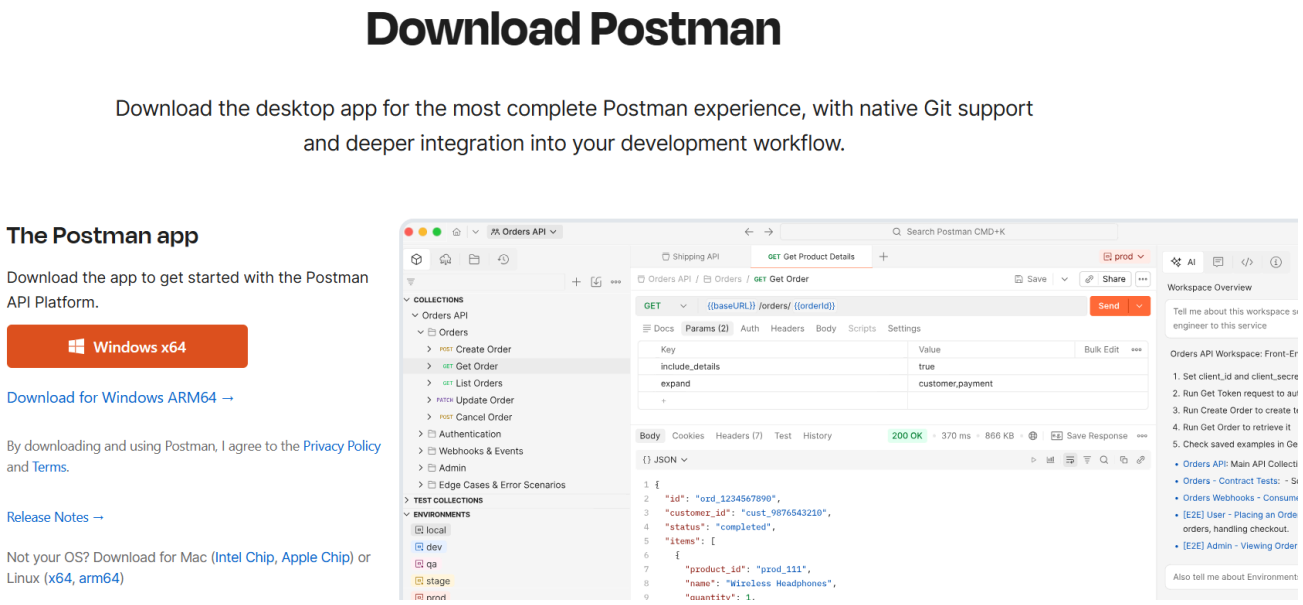
POSTMAN

Установка

Postman доступен в двух вариантах:

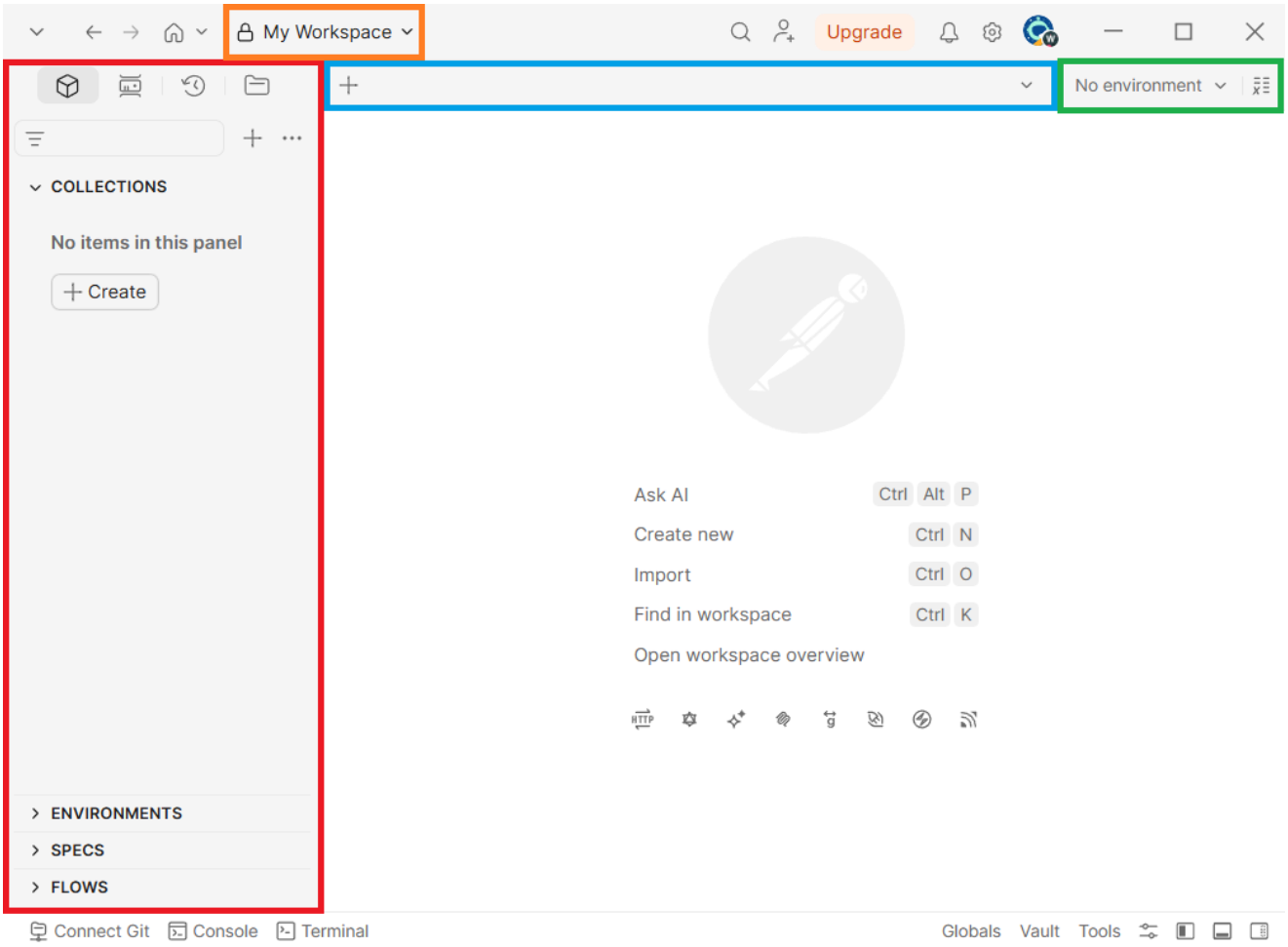
- **Настольное приложение (Desktop App)** — рекомендуемый вариант. Загружается с официального сайта postman.com и устанавливается на операционные системы Windows, macOS и Linux.
- **Веб-версия (Web App)** — доступна без установки, однако имеет ограниченный функционал при работе с локальными серверами.

Для работы с локально запущенным API рекомендуется использовать настольное приложение.



Интерфейс

После установки и входа в учётную запись пользователю открывается главный интерфейс Postman.

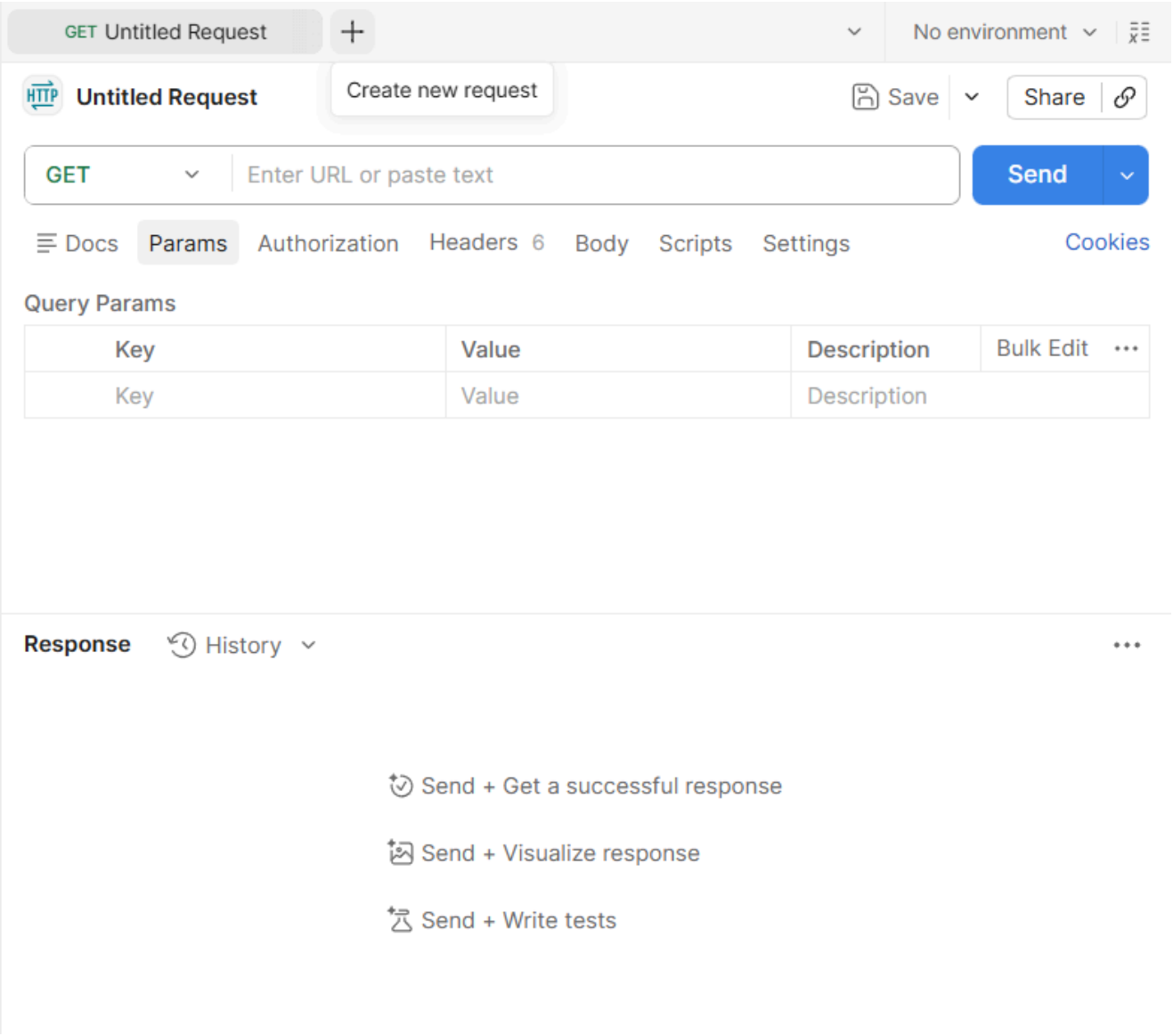


Основные элементы интерфейса:

- **Sidebar (боковая панель)** — содержит коллекции, историю запросов и настройки окружений.
- **Tabs (вкладки запросов)** — область, в которой открываются и редактируются запросы.
- **Workspace (рабочее пространство)** — логическая единица организации работы. Рабочее пространство может быть личным (Personal) или командным (Team).
- **Environment selector (выбор окружения)** — расположен в правом верхнем углу; позволяет быстро переключаться между настроенными окружениями.

2. Создание запроса

Для создания нового запроса необходимо нажать кнопку **«+»** на панели вкладок или воспользоваться сочетанием клавиш `Ctrl+T`. Откроется вкладка нового запроса.



В верхней части вкладки расположены:

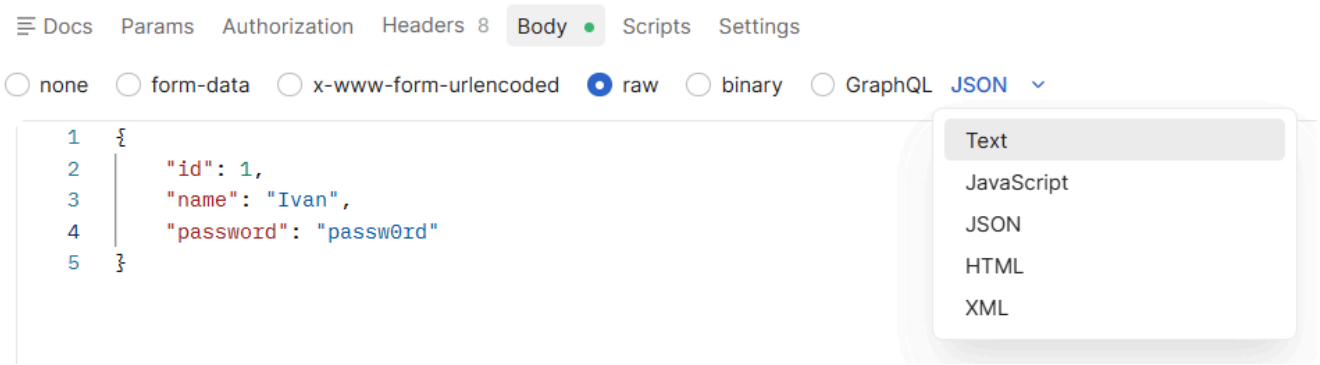
- **Выпадающий список метода HTTP** — позволяет выбрать метод: `GET`, `POST`, `PUT`, `PATCH`, `DELETE` и другие.
- **Поле URL** — адрес запроса.
- **Кнопка «Send»** — отправка запроса.

Настройка параметров запроса

Ниже строки URL расположены вкладки с параметрами:

- **Params** — параметры строки запроса (query string). При заполнении данной вкладки Postman автоматически формирует URL вида `?key=value&key2=value2`.
- **Authorization** — настройка авторизации (рассматривается в разделе 6).

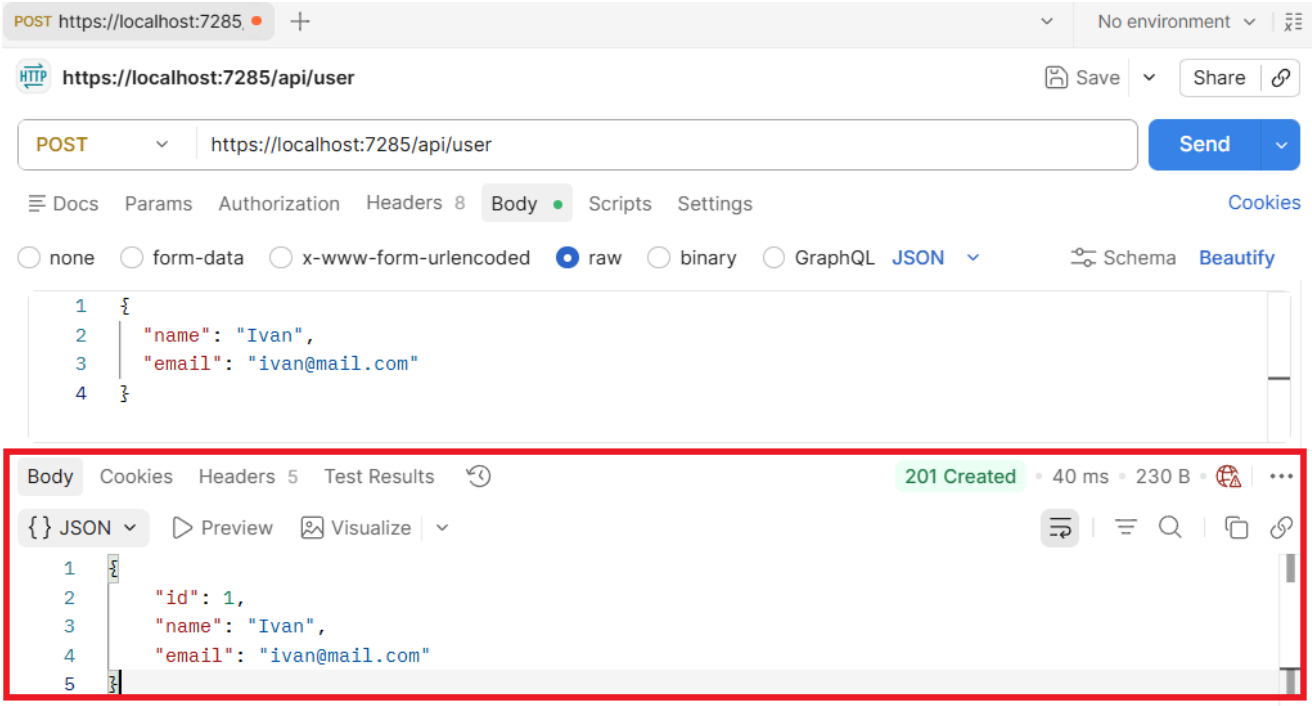
- **Headers** — заголовки запроса. Часть заголовков (например, `Content-Type`) Postman подставляет автоматически.
- **Body** — тело запроса. Активно для методов `POST`, `PUT`, `PATCH`. Режим **raw** → **JSON** используется для отправки JSON-данных.



Анализ ответа

После отправки запроса в нижней части вкладки отображается панель ответа. Она содержит следующие элементы:

- **Статус-код** (например, `200 OK`, `201 Created`, `404 Not Found`) — отображается в верхней строке панели ответа.
- **Время выполнения** (Time) — время от отправки запроса до получения ответа в миллисекундах.
- **Размер ответа** (Size) — объём полученных данных.
- **Вкладка Body** — тело ответа. Доступны режимы Pretty (форматированный JSON), Raw (исходный текст) и Preview (HTML-рендер).
- **Вкладка Headers** — заголовки ответа.



3. Коллекции

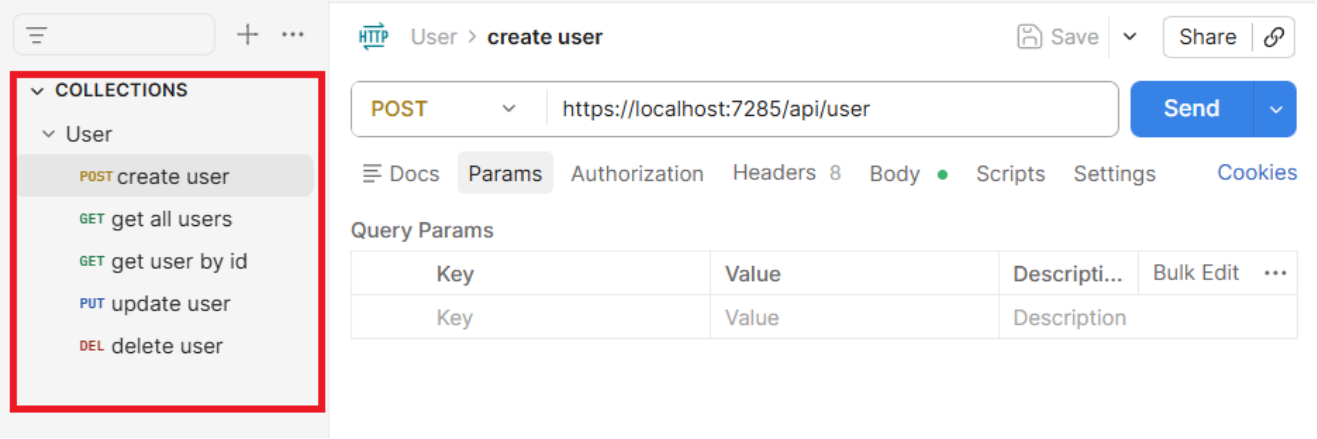
Коллекция — это именованная группа запросов, объединённых по смыслу или принадлежности к одному API. Коллекции являются основным способом организации работы в Postman.

Использование коллекций позволяет:

- хранить все запросы к конкретному API в одном месте;
- группировать запросы по папкам (например, по модулям или ресурсам API);
- применять общие настройки авторизации и переменных ко всем запросам коллекции;
- генерировать документацию API на основе коллекции;
- обмениваться набором запросов внутри команды.

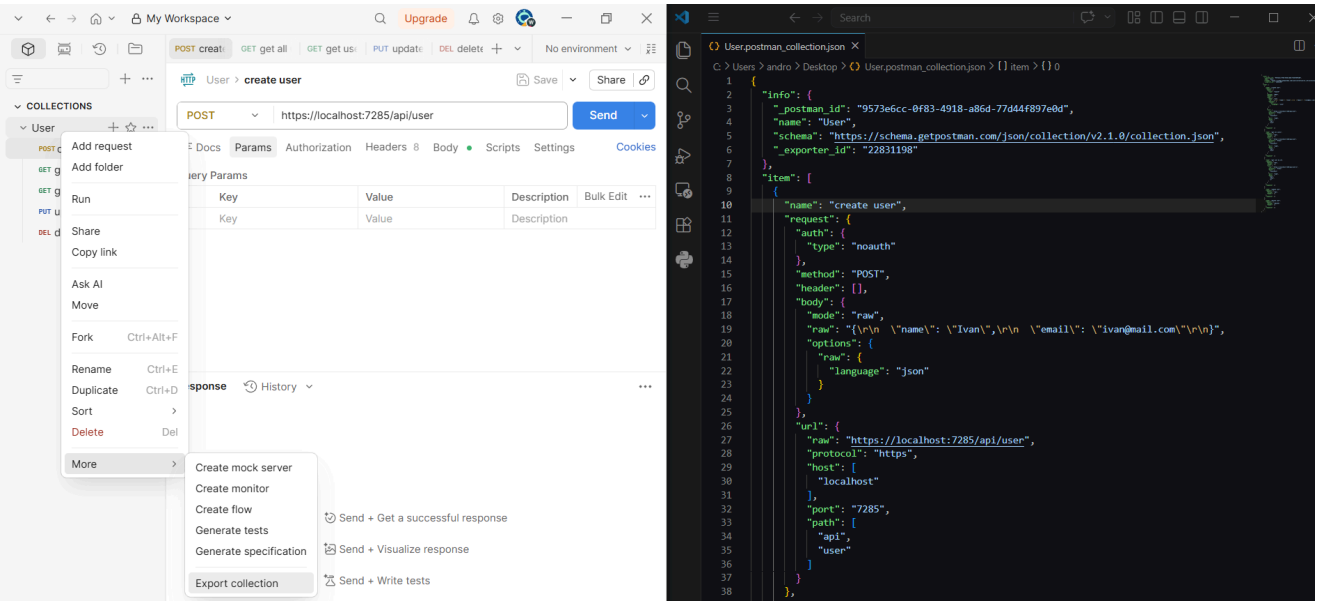
Создание коллекции и добавление запросов

Для создания коллекции необходимо выбрать **Collection** на боковой панели и нажать кнопку **Create**. После присвоения имени коллекции можно добавлять в неё запросы.



Экспорт и импорт коллекции

Коллекция может быть экспортирована в формате JSON-файла через контекстное меню коллекции → **«Export»**. Полученный файл можно передать коллеге, добавить в систему контроля версий (Git) или импортировать на другом рабочем месте через **«Import»**.



Таким образом, коллекция обеспечивает воспроизводимость набора тестовых запросов в любой среде и возможность командной работы.

4. Переменные и окружения

При разработке API часто возникает ситуация, когда одни и те же запросы необходимо выполнять в разных средах: на локальной машине разработчика, на тестовом сервере и в продуктивной среде. Если базовый URL прописан явно в каждом запросе, смена окружения требует ручного изменения множества значений, что неудобно и чревато ошибками.

Postman решает эту проблему с помощью **переменных**.

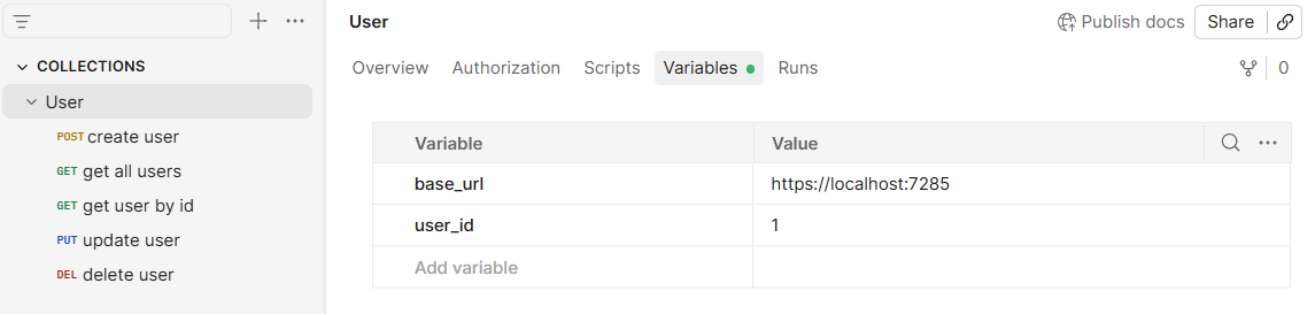
Типы переменных

В Postman существует несколько уровней переменных, которые различаются областью видимости:

Уровень	Область видимости	Приоритет
Global	Все коллекции во всех окружениях	Низкий
Collection	Все запросы данной коллекции	Средний
Environment	Активное окружение	Высокий
Local	Только текущий скрипт	Наивысший

Переменная с более высоким приоритетом переопределяет переменную с тем же именем на нижнем уровне.

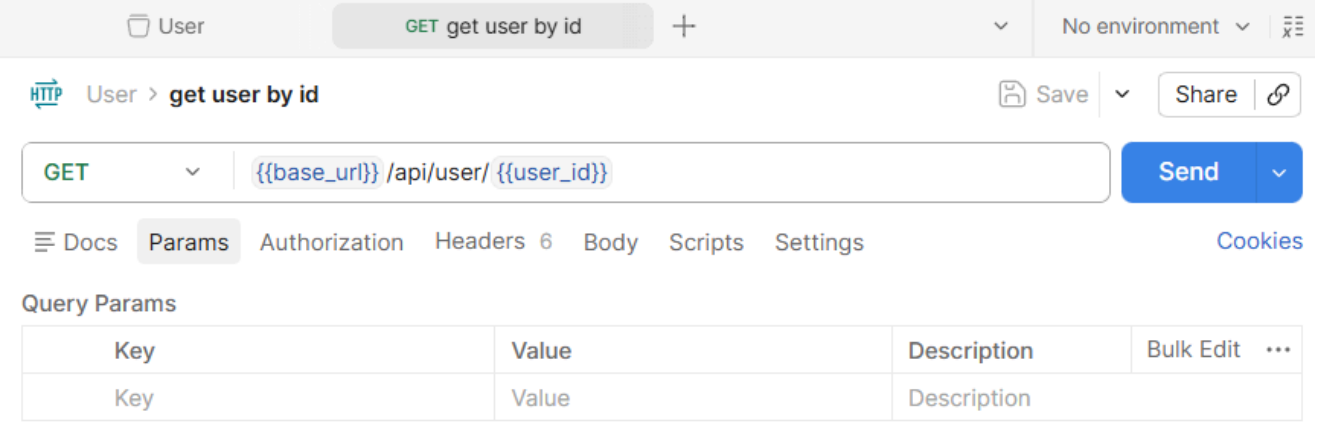
Использование переменных в запросах



Переменная вставляется в любое поле запроса с помощью синтаксиса двойных фигурных скобок: `{{имя_переменной}}`.

Postman подсвечивает переменные: оранжевым — если переменная не определена в текущем окружении, зелёным — если определена.

Пример использования переменной в URL:



Создание окружений

Для создания окружения необходимо перейти в раздел **«Environments»** в боковой панели и нажать **«+»**. В окружении задаются пары «ключ — значение».

Пример настройки двух окружений:

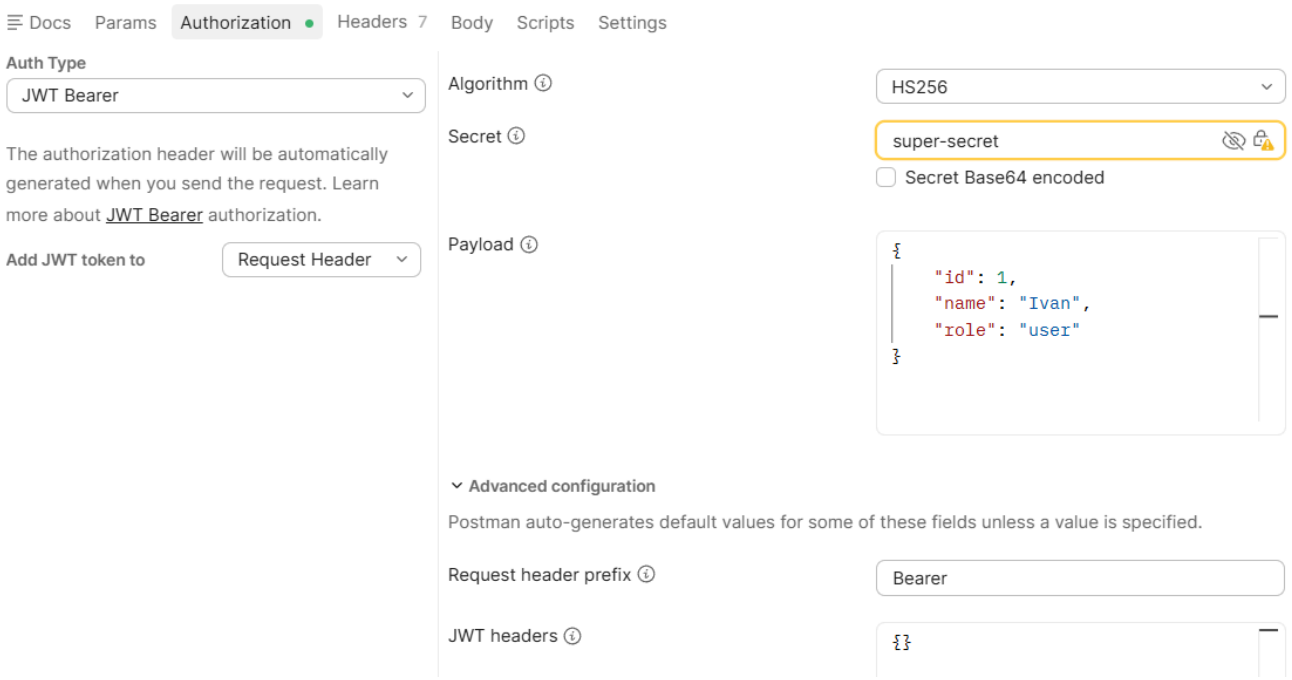
Переменная	Development	Production
`base_url`	`http://localhost:5000`	`https://api.myapp.com`
`api_version`	`v1`	`v1`

После создания окружений переключение между ними выполняется через выпадающий список в правом верхнем углу интерфейса. При этом все запросы в коллекции автоматически начинают использовать значения из выбранного окружения.

5. Авторизация

Postman поддерживает все распространённые схемы авторизации HTTP-запросов. Настройка осуществляется на вкладке **«Authorization»** запроса или коллекции:

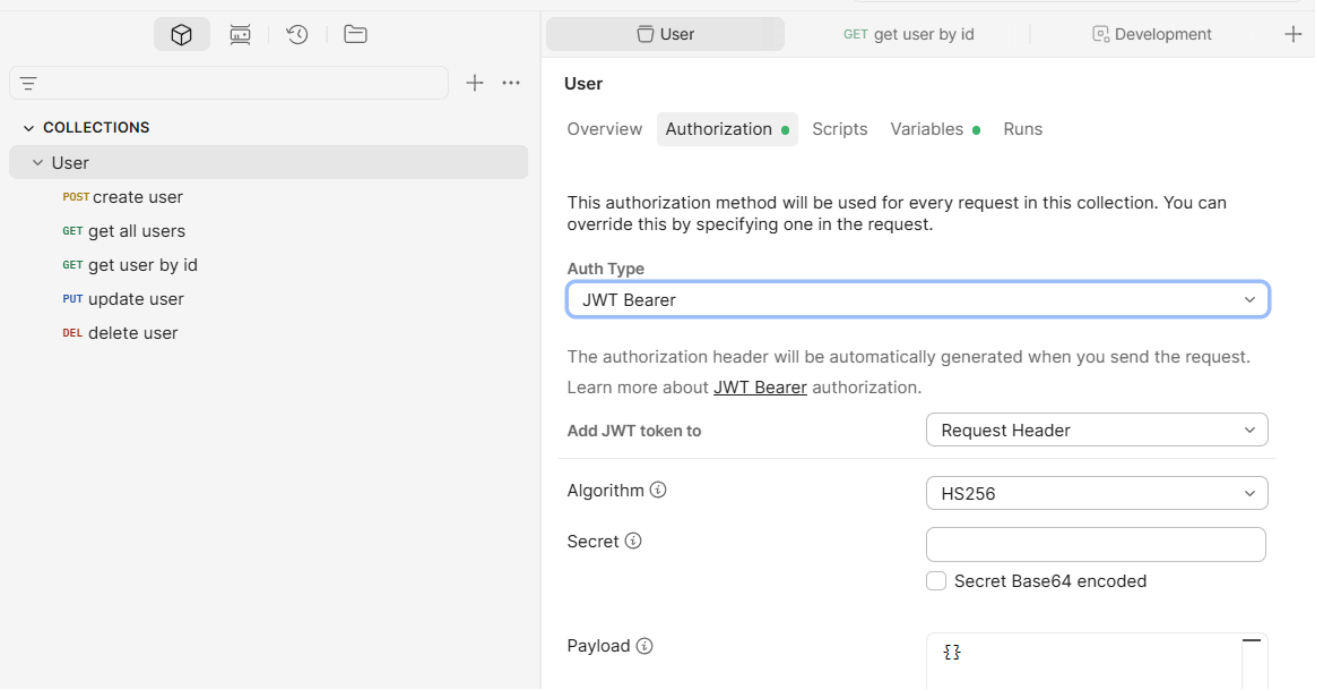
- **Bearer Token** — передача JWT или иного токена в заголовке `Authorization: Bearer <token>`.
- **Basic Auth** — передача имени пользователя и пароля в заголовке в кодировке Base64.
- **API Key** — передача ключа в заголовке или в параметрах строки запроса.
- **OAuth 2.0** — полноценный протокола авторизации OAuth с получением токена.



Авторизация на уровне коллекции

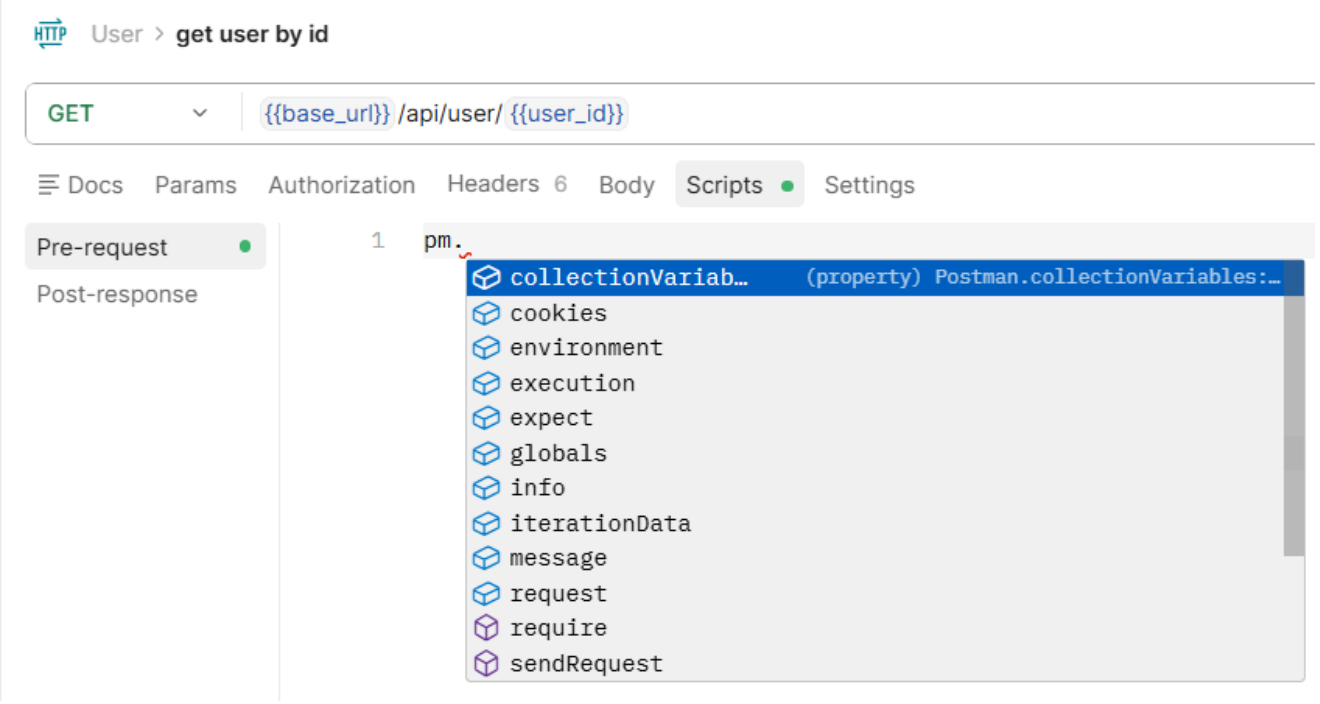
Ключевая особенность Postman — возможность задать авторизацию один раз на уровне всей коллекции. В настройках коллекции (вкладка **«Authorization»**) указывается тип и параметры авторизации. Во всех запросах коллекции в поле Authorization выбирается значение **«Inherit auth from parent»** — и каждый запрос автоматически использует токен коллекции.

Это исключает необходимость дублировать токен в каждом запросе и упрощает его обновление: достаточно изменить значение в одном месте.



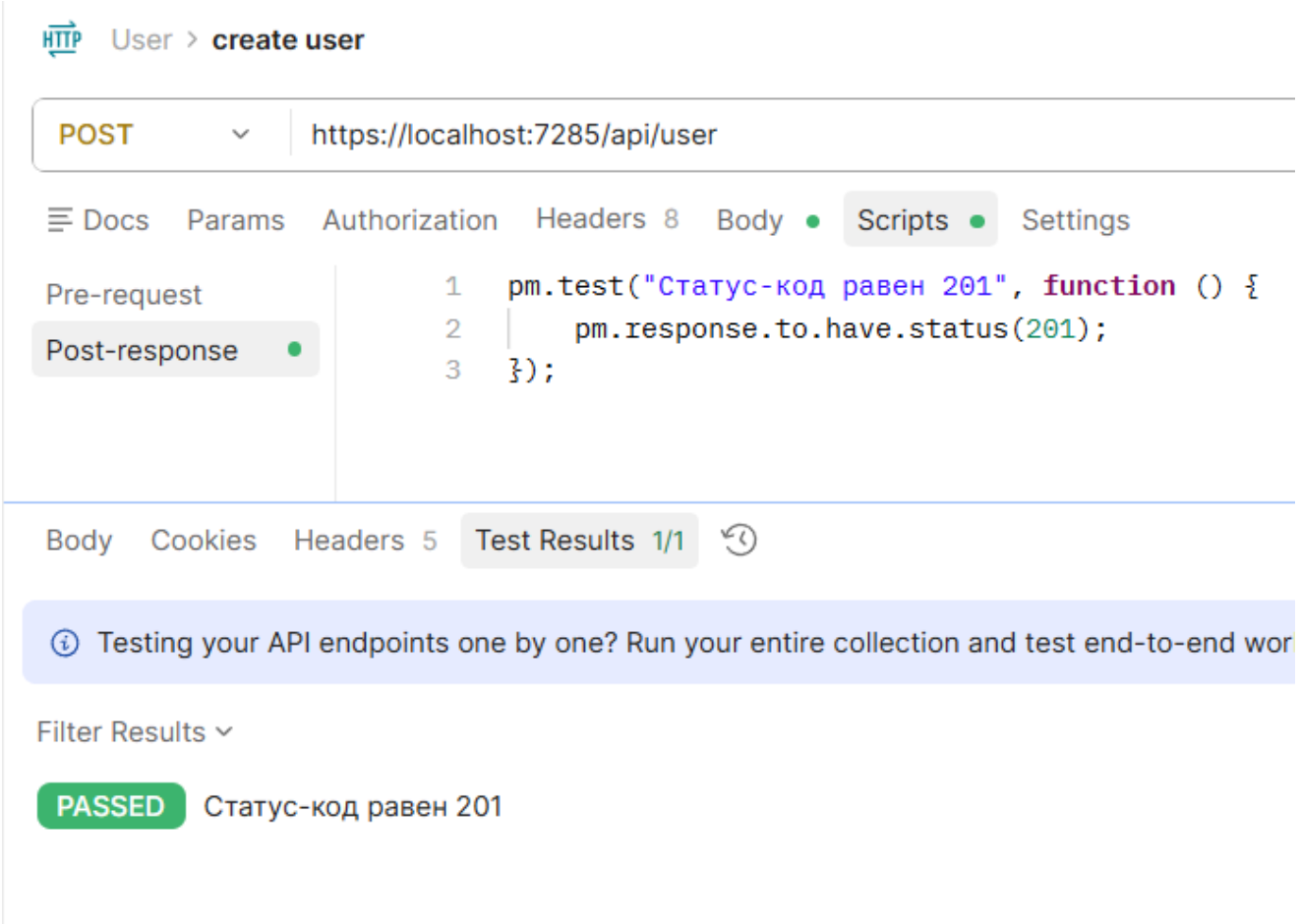
6. Автоматические тесты

Postman позволяет прикреплять к каждому запросу JavaScript-скрипты, которые автоматически выполняются после получения ответа. Скрипты размещаются на вкладке **«Scripts»** запроса. Для написания скриптов предоставляется библиотека `pm` (Postman), которая предоставляет API для работы с запросом, ответом и переменными.



Проверка статус-кода

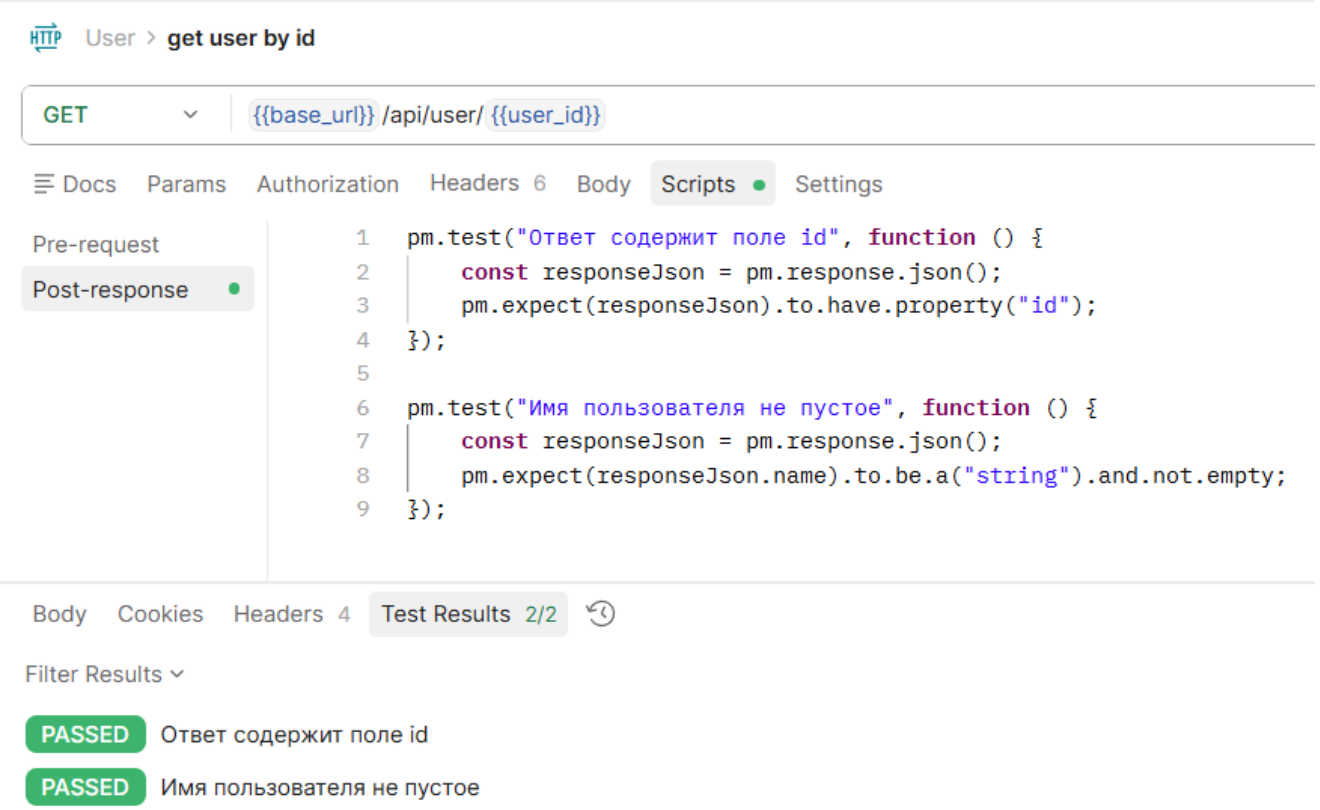
Наиболее частая проверка — соответствие статус-кода ожидаемому значению:



Функция `pm.test()` принимает два аргумента: текстовое описание теста и функцию-проверку. В случае провала проверки тест помечается как неуспешный.

Проверка тела ответа

Проверка содержимого JSON-ответа выполняется следующим образом:



Сохранение значений из ответа в переменные

Одной из наиболее мощных возможностей является извлечение данных из ответа и сохранение их в переменные для использования в последующих запросах. Типичный пример — получение токена авторизации после запроса на аутентификацию:

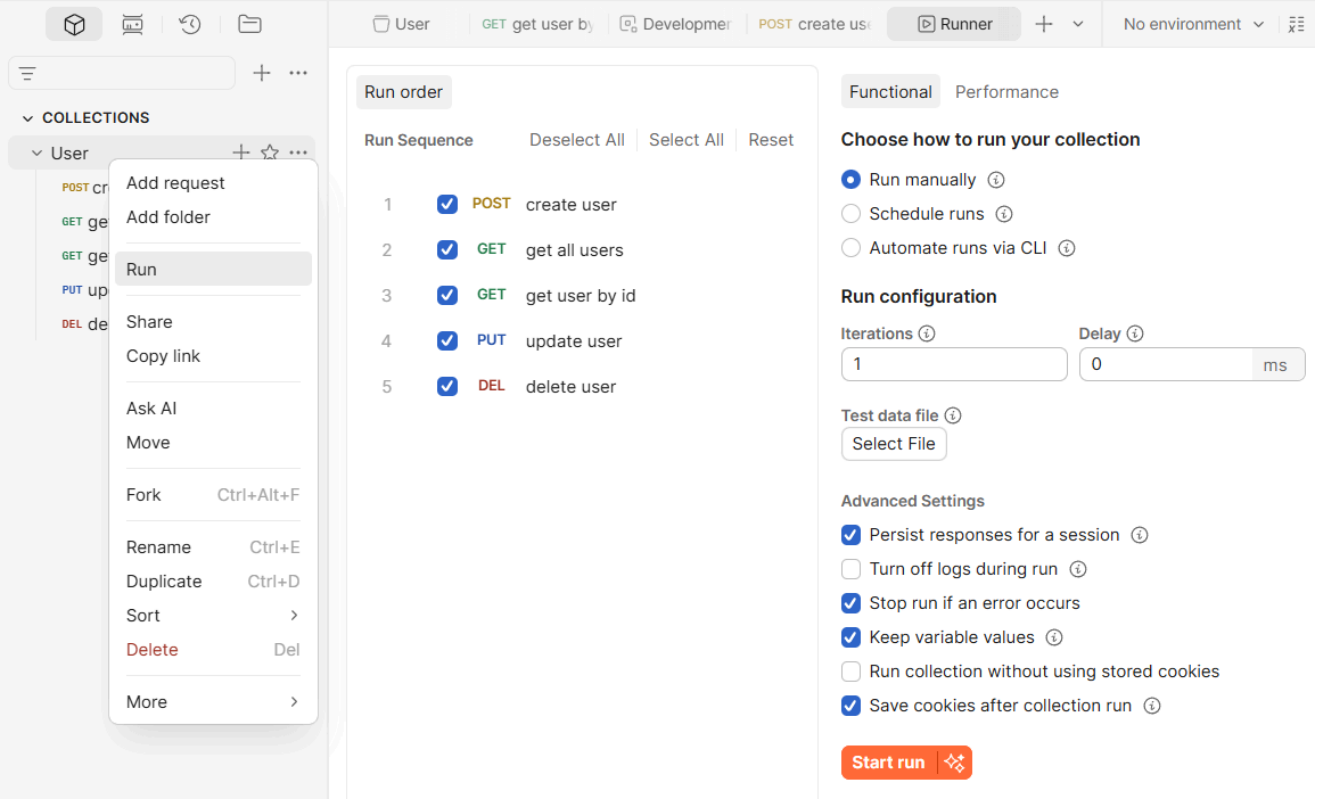
```
// Скрипт запроса POST /auth/login
pm.test("Аутентификация успешна", function () {
    pm.response.to.have.status(200);
});

const responseJson = pm.response.json();
pm.collectionVariables.set("auth_token", responseJson.token);
```

После выполнения данного скрипта переменная `{{auth_token}}` становится доступна во всех последующих запросах коллекции.

7. Collection Runner

Collection Runner — это встроенный инструмент Postman, предназначенный для автоматического последовательного выполнения всех запросов коллекции или выбранной папки. Он позволяет запустить полный набор тестов за одно действие и получить сводный отчёт о результатах.

Для запуска необходимо открыть коллекцию и нажать кнопку **«Run»**.



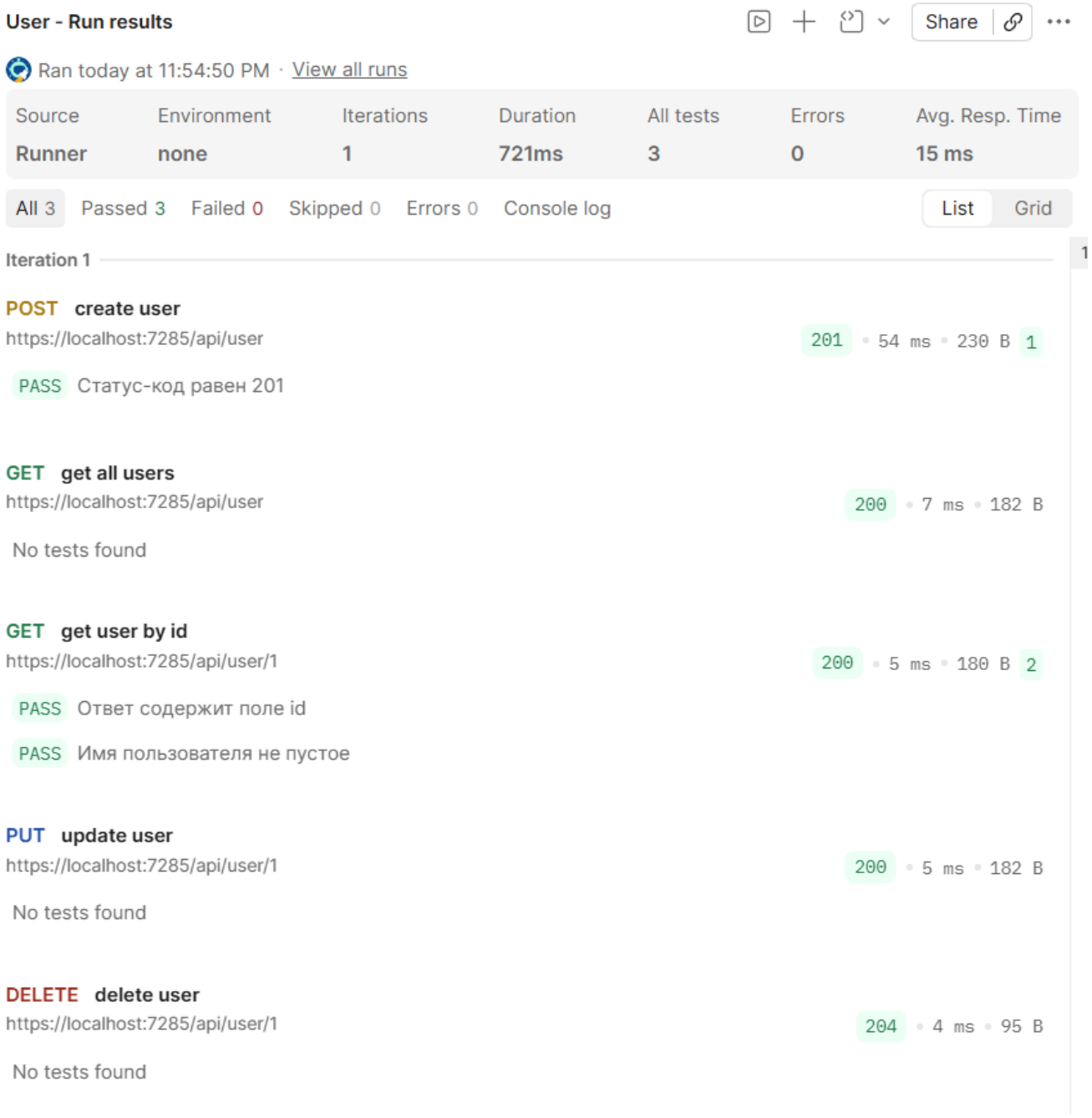
Параметры запуска

В интерфейсе Collection Runner доступны следующие настройки:

- **Iterations** — количество повторений. При значении больше 1 весь набор запросов выполняется заданное число раз подряд.
- **Delay** — задержка в миллисекундах между запросами.

Анализ результатов

После завершения прогона отображается сводная панель результатов: для каждого запроса указывается статус (Pass/Fail) каждого теста, фактический статус-код ответа, а также время выполнения.



Применение для нагрузочного тестирования

В Collection Runner есть поддержка нагрузочного тестирования через вкладку **«Performance»**. Это позволяет проводить базовое нагрузочное тестирование API непосредственно в интерфейсе Postman, без подключения сторонних инструментов.

Настройка теста выполняется на вкладке Performance в Collection Runner. Доступны следующие параметры:

- **Load profile** — профиль нагрузки. Вариант **Fixed** задаёт постоянное число виртуальных пользователей на протяжении всего теста; доступны также профили с нарастающей нагрузкой.
- **Virtual users** — количество виртуальных пользователей, одновременно выполняющих запросы коллекции.
- **Test duration** — продолжительность теста в минутах.

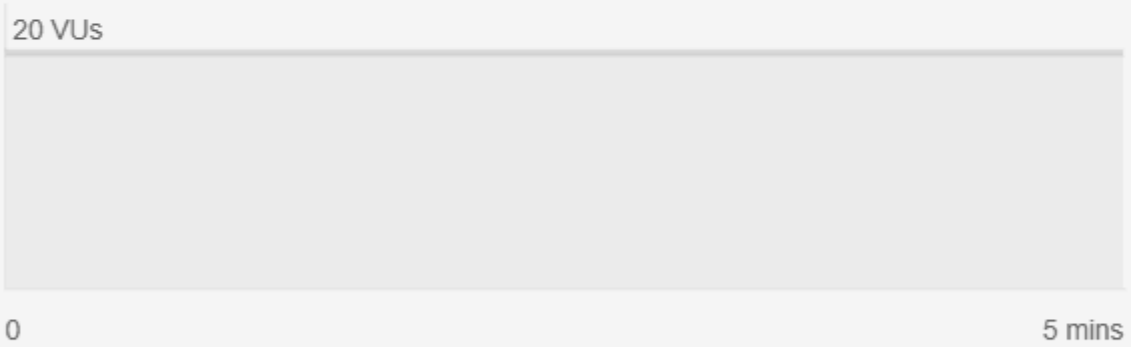
Functional | Performance

Choose how to run your performance test

- (●) In the app
  Run this performance test in the app.
- ( ) Via the CLI
  Configure CLI command to run on your build pipeline.

Set up your performance test

Load profile	Virtual users	Test duration
Fixed	20	5 mins



20 virtual users run for 5 minutes, each executing all requests sequentially.

Data file

Select file

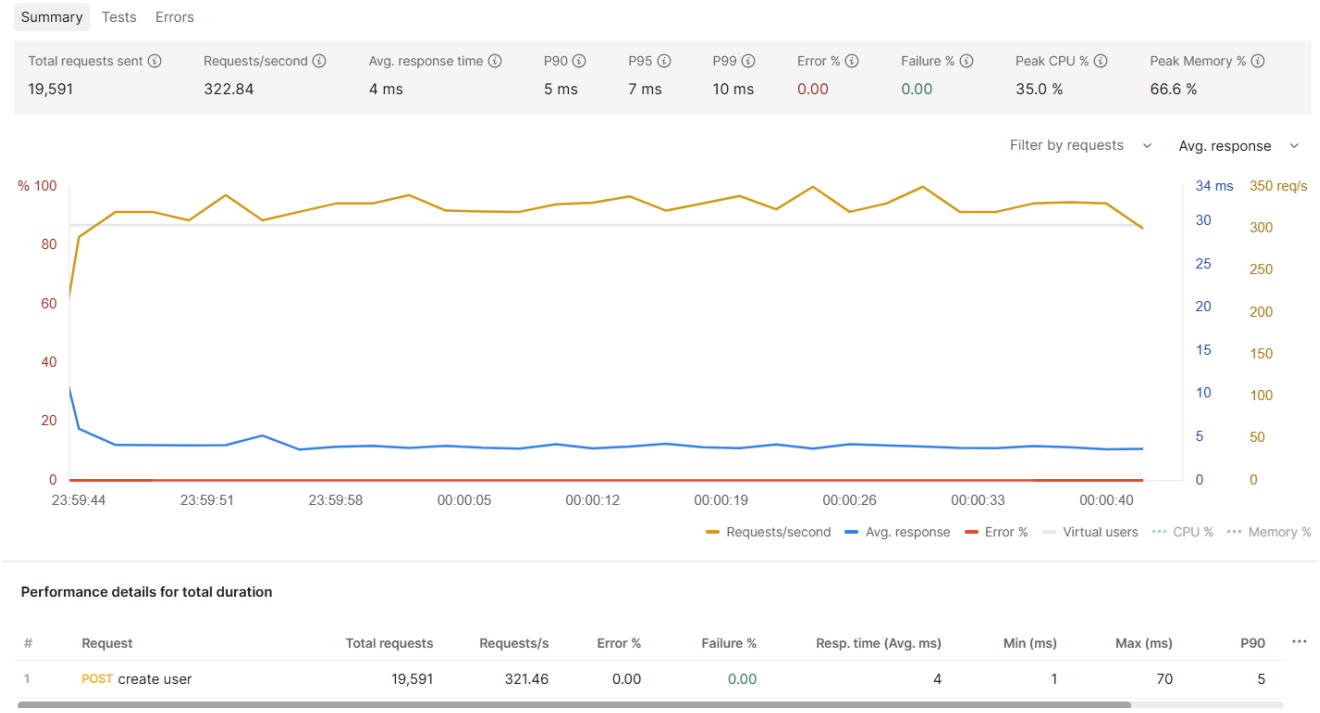
› Pass test if...

Run

Результаты теста отображаются в режиме реального времени на сводной панели Summary и включают следующие метрики:

- **Total requests sent** — общее количество отправленных запросов за время теста.
- **Requests/second** — средняя пропускная способность.
- **Avg. response time** — среднее время отклика в миллисекундах.
- **P90 / P95 / P99** — перцентили времени отклика, позволяющие оценить поведение системы под нагрузкой в худших сценариях.
- **Error % / Failure %** — доля запросов, завершившихся ошибкой на уровне HTTP или на уровне тестовых проверок.
- **Peak CPU % / Peak Memory %** — пиковая загрузка ресурсов сервера (при наличии интеграции с агентом мониторинга).

На графике в реальном времени отображаются кривые пропускной способности (Requests/second), среднего времени отклика (Avg. response) и процента ошибок (Error %).



Практическое задание (40)

Что нужно сделать:

В рамках данного задания необходимо провести полное тестирование ранее разработанного REST API для приложения TodoApp с использованием Postman. Результаты работы оформляются в виде структурированного отчёта и сохраняются в репозитории проекта. Для выполнения установите Postman и зарегистрируйтесь в нем.

1. Коллекция и организация запросов

Создать коллекцию **TodoApp API** и организовать запросы по папкам в соответствии с ресурсами API. Например:

- `Auth` — регистрация, вход, выход
- `Todos` — CRUD-операции над задачами

Коллекция должна содержать запросы для **каждого** реализованного эндпоинта.

2. Переменные и окружения

Создать как минимум одно окружение (например, `Development`) со следующими переменными:

- `{{base_url}}` — базовый адрес API (`http://localhost:XXXX`)
- `{{auth_token}}` — токен авторизации (заполняется автоматически скриптом после логина)
- Любые другие переменные, необходимые для работы запросов (например, `{{todo_id}}`)

Все запросы коллекции должны использовать переменные — жёстко заданные URL и токены не допускаются.

3. Автоматические тесты

Для каждого запроса написать тестовые скрипты на вкладке **Tests**. Минимальные требования:

- Проверка корректного статус-кода для каждого запроса
- Проверка структуры и содержимого тела ответа (наличие обязательных полей)
- Для запросов создания ресурсов — извлечение `id` созданного объекта и сохранение в переменную для использования в последующих запросах

4. Запуск через Collection Runner

Выстроить порядок запросов в коллекции таким образом, чтобы их последовательное выполнение через **Collection Runner** образовывало связный сценарий:

1. Регистрация / вход — получение токена
2. Создание задач
3. Получение списка задач
4. Обновление задачи
5. Удаление задачи

Все тесты в Collection Runner должны завершиться со статусом **Pass**.

5. Нагрузочное тестирование

Провести нагрузочный тест одного из эндпоинтов (наиболее нагруженные, например `GET /todos` или `POST /todos`) с использованием вкладки **Performance** в Collection Runner.